

Kolumnowe bazy danych cz. II

Zaawansowana analityka i dowód wydajności

Imię i nazwisko: Marek Małek, Mateusz Lampert

Grupa: 4

Cel ćwiczenia

Po tym laboratorium będziesz potrafił:

- zbudować lejek konwersji i zinterpretować odpływ między jego etapami,
- użyć funkcji okna do rankingu i analizy trendu przychodów w czasie,
- podzielić użytkowników na segmenty metodą RFM i wyciągnąć z tego wnioski biznesowe,
- zmierzyć i wyjaśnić różnicę wydajności między ClickHouse a PostgreSQL i powiedzieć, skąd ona wynika.

Laboratorium ma celowo prostą strukturę: **zadania 1–3 to analityka w ClickHouse, zadanie 4 to dowód** eksperyment porównawczy oparty na zapytaniach, które właśnie napisałeś.

Zanim zaczniesz - przeczytaj to uważnie

Której bazy używamy i kiedy

Zadanie	Baza danych
0. Gotowość	obie — sprawdzasz środowisko
1. Lejek konwersji	ClickHouse
2. Funkcje okna	Jedna baza do wyboru
3. Segmentacja RFM	Do wyboru ale ClickHouse lepiej
4. Benchmark i wnioski	obie — porównanie obowiązkowe

W tym laboratorium ClickHouse jest bazą wiodącą. PostgreSQL pojawia się w zadaniu 4 jako punkt odniesienia — po to, żeby różnicę wydajności zmierzyć, nie zakładać.

Jak korzystać ze ściąg

Do zajęć dołączona jest ściąg `sql_postgresql_clickhouse_sciaga.pdf`. Zawiera składnię i podpowiedzi do przykładów dla każdego zadania. Korzystaj z niej jak z dokumentacji - żeby sprawdzić składnię funkcji, nie żeby skopiować rozwiązanie. Numer sekcji ściąg podany jest przy każdym zadaniu.

Oceniane są interpretacja i komentarz. Sam fakt uruchomienia zapytania nie jest rozwiązaniem.

Sprawozdanie

Oddaj jako PDF albo Markdown. Dla każdego zadania dołącz:

- kod zapytania,
- wynik - tabela lub zrzut ekranu,
- komentarz - pełnymi zdaniami, nie listą słów.

Termin oddania: do końca dnia poprzedzającego kolejne zajęcia.

Punktacja: razem 10 pkt.

Założenie startowe

Środowisko Docker działa, tabela events jest dostępna w obu bazach. Jeżeli tabela nie jest widoczna w bieżącym kontekście, użyj pełnej nazwy:

- `public.events` w PostgreSQL,
- `ds_lab.events` w ClickHouse.

0. Gotowość do zajęć — warunek konieczny (0 pkt)

Pokaż, że środowisko działa: kontenery `postgres` i `clickhouse` mają status `Up`, tabela `events` jest widoczna w obu bazach, połączenie z klienta SQL działa.

Brak gotowości środowiska uniemożliwia wykonanie dalszych zadań.

1. Lejek konwersji — 2 pkt

Dlaczego to robimy

W e-commerce każda sesja przechodzi przez etapy: wyświetlenie → koszyk → zakup. Na każdym etapie część sesji odpada. **Lejek konwersji** mierzy, ile sesji przechodzi przez każdy etap i gdzie odpływ jest największy - to fundamentalny wskaźnik analityki e-commerce. Przy okazji zobaczysz, jak ClickHouse upraszcza agregaty warunkowe dzięki `countIf` zamiast klasycznego `CASE WHEN`. (zobacz odpowiednie sekcje w ściągde)

Wykonaj w ClickHouse

Dla każdej sesji ustal, czy wystąpiło w niej zdarzenie view, cart i purchase. Na tej podstawie policz dla całego zbioru:

- liczbę sesji z view,
- liczbę sesji z cart,
- liczbę sesji z purchase,
- wskaźnik cart / view — jaka część sesji z view dotarła do cart,
- wskaźnik purchase / cart — jaka część sesji z cart zakończyła się zakupem,
- wskaźnik purchase / view — ogólna konwersja od pierwszego kontaktu do zakupu.

Następnie powtórz analizę w przekroju device.

Zapytanie startowe

```
-- Krok 1: flagi na poziomie sesji
WITH session_flags AS (
    SELECT
        session_id,
        countIf(event_type = 'view')      > 0 AS has_view,
        countIf(event_type = 'add_to_cart') > 0 AS has_cart,
        countIf(event_type = 'purchase') > 0 AS has_purchase
    FROM events
    GROUP BY session_id
)
-- Krok 2: agregacja do poziomu całego zbioru
SELECT
    countIf(has_view)      AS view_sessions,
    countIf(has_cart)      AS cart_sessions,
    countIf(has_purchase) AS purchase_sessions,
    countIf(has_cart) / nullIf(countIf(has_view), 0) AS cart_per_view,
    countIf(has_purchase) / nullIf(countIf(has_cart), 0) AS purchase_per_cart,
    countIf(has_purchase) / nullIf(countIf(has_view), 0) AS purchase_per_view
FROM session_flags;
```

Zbuduj analogicznie wersję z GROUP BY device.

W komentarzu napisz

- Na którym etapie lejka odpływ jest największy i co to oznacza dla biznesu — gdzie sklep traci klientów?
- Czy lejek różni się między urządzeniami? Jeśli tak, co może być tego przyczyną?
- countIf zastępuje klasyczny wzorec MAX(CASE WHEN ... THEN 1 ELSE 0 END) ze standardowego SQL. W 2–3 zdaniach wyjaśnij, na czym polega różnica w podejściu i dlaczego wersja ClickHouse jest krótsza.

Rozwiązanie

Grupowanie po device:`

```
WITH session_flags AS (SELECT session_id,
                             device,
                             countIf(event_type = 'view') > 0      AS has_view,
                             countIf(event_type = 'add_to_cart') > 0 AS has_cart,
                             countIf(event_type = 'purchase') > 0   AS has_purchase
                        FROM events
                        GROUP BY device, session_id)

SELECT device,
    countIf(has_view)      AS view_sessions,
    countIf(has_cart)      AS cart_sessions,
    countIf(has_purchase) AS purchase_sessions,
    countIf(has_cart) / nullIf(countIf(has_view), 0) AS cart_per_view,
    countIf(has_purchase) / nullIf(countIf(has_cart), 0) AS purchase_per_cart,
    countIf(has_purchase) / nullIf(countIf(has_view), 0) AS purchase_per_view
FROM session_flags
GROUP BY device
```

Wyniki zapytań:

- Lejek ogólny:

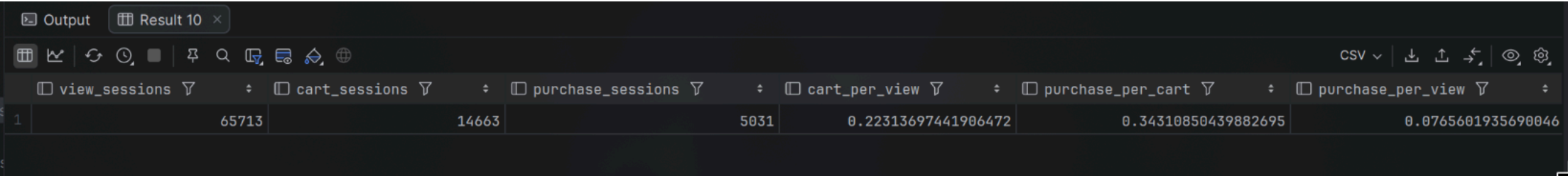


Figure 1: alt text

- Lejek w przekroju urządzeń:

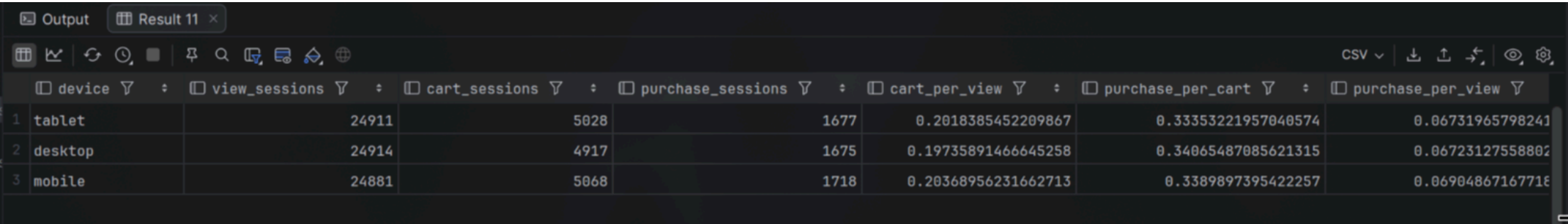


Figure 2: alt text

Komentarz:

- największy procentowo odpływ jest między etapami view a cart, tylko 7% klientów, którzy zobaczyli produkt, dodali go do koszyka. To oznacza, że sklep traci większość potencjalnych klientów na etapie zainteresowania produktem - być może strona produktu nie jest wystarczająco przekonująca, brakuje informacji, zdjęć, recenzji, albo ceny są zbyt wysokie.
- lejek nieznacznie różni się między urządzeniami, ale ogólnie jest taki sam. Na desktopie jest najmniej sesji cart (4917 vs 5028, 5068). Na mobile jest najwięcej sesji purchase (1718 vs 1677, 1675).
- countIf to funkcja agregująca, która zlicza tylko te wiersze, które spełniają warunek. W klasycznym SQL musielibyśmy użyć CASE WHEN do stworzenia flagi 0/1, a następnie zliczyć sumę tych flag. countIf pozwala zrobić to bezpośrednio, co skraca i upraszcza zapytanie.

2. Funkcje okna: rankingi i trend przychodów 2 pkt

Dlaczego to robimy

Zwykle GROUP BY zwija dane - dostajesz jeden wiersz na grupę. **Funkcje okna** liczą wartości w kontekście innych wierszy bez zwijania wyników: ranking krajów, suma narastająca, zmiana dzień do dnia bez zagnieżdżonych podzapytań.

Wybierz jedną bazę i napisz w niej obie części

Zaznacz w sprawozdaniu, którą bazę wybrałeś.

Część A - Ranking krajów

Policz łączny przychód z zakupów dla każdego kraju i nadaj krajom ranking według przychodu. Użyj RANK() albo DENSE_RANK().

Wynik powinien zawierać kolumny: country, revenue, rank_no.

Wskazówka: funkcję okna możesz zastosować bezpośrednio w SELECT obok agregacji - nie musisz pisać podzapytania ani CTE.

Część B - Narastający przychód i zmiana dzień do dnia

Policz dzienny przychód ze zdarzeń purchase. Następnie w tym samym zapytaniu oblicz:

- sumę narastającą - cumulative_revenue,
- przychód z poprzedniego dnia - prev_day_revenue,
- zmianę procentową względem poprzedniego dnia - pct_change.

Wynik powinien zawierać kolumny: day, daily_revenue, cumulative_revenue, prev_day_revenue, pct_change.

Wskazówki składniowe

Element	PostgreSQL	ClickHouse
Data	DATE(event_time)	toDate(event_time)
Suma narastająca	SUM(x) OVER (ORDER BY day)	sum(x) OVER (ORDER BY day ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)
Poprzednia wartość	LAG(x, 1) OVER (ORDER BY day)	lagInFrame(x, 1) OVER (ORDER BY day ROWS ...)

Wskazówka: zbuduj zapytanie w dwóch krokach - najpierw CTE z dziennym przychodem (GROUP BY day), a dopiero do niego dołącz funkcje okna.

W komentarzu napisz

- Czy suma narastająca rośnie równomiernie czy skokowo?

- Czy widać konkretny dzień z wyraźną zmianą - ile wyniósł wzrost lub spadek w procentach?
- Co było dla Ciebie nowe w funkcjach okna i co sprawiło największą trudność?

Rozwiązanie

W rozwiązaniu wykorzystana została baza Clickhouse.

Zadanie 2a)

Zapytanie:

```
select
    country,
    sum(price * quantity) as revenue,
    rank() over (order by sum(price * quantity) desc ) as rank_no
from events
where event_type = 'purchase'
group by country
order by rank_no;
```

Wyniki:

	country	revenue	rank_no
1	GB	189364.49	1
2	IT	187879.44	2
3	NL	185101.92999999996	3
4	PL	173735.24999999997	4
5	FR	172757.88	5
6	NO	171390.43000000002	6
7	US	167135.53	7
8	ES	164466.48	8
9	DE	162931.26	9
10	SE	159174.65	10

Figure 3: alt text

Komentarz:

- krajem o najwyższym łącznym przychodzie okazała się Wielka Brytania, natomiast krajem o najniższym łącznym przychodzie Szwecja.
- w tym konkretnym zastosowaniu nie ma znaczenia czy wykorzystaliśmy rank()/denserank(), ponieważ nie występują remisy.

Zadanie 2b)

Zapytanie:

```
with daily as (
    select toDate(event_time) AS day, sum(price * quantity) AS daily_revenue
    from events
    where event_type = 'purchase'
```

```

    group by day
)
select day, daily_revenue,
       sum(daily_revenue) over (order by day rows between unbounded preceding and current row) as cumulative_revenue,
       lagInFrame(toNullable(daily_revenue), 1, null) over (order by day rows between unbounded preceding and current row) as prev_day_revenue,
       round((daily_revenue - prev_day_revenue) / prev_day_revenue * 100, 2) as pct_change
from daily;
```

Wyniki:

	📅 day 📄	📊 daily_revenue 📄	📊 cumulative_revenue 📄	📊 prev_day_revenue 📄	📊 pct_change 📄
1	2025-01-01	8095.42	8095.42	<null>	<null>
2	2025-01-02	10359.45	18454.870000000003	8095.42	27.97
3	2025-01-03	8693.080000000002	27147.950000000004	10359.45	-16.09
4	2025-01-04	9883.41	37031.36	8693.080000000002	13.69
5	2025-01-05	5602.400000000001	42633.76	9883.41	-43.32
6	2025-01-06	10984.43	53618.19	5602.400000000001	96.07
7	2025-01-07	11007.97	64626.16	10984.43	0.21
8	2025-01-08	9961.59	74587.75	11007.97	-9.51
9	2025-01-09	6287.560000000001	80875.31	9961.59	-36.88
10	2025-01-10	12643.200000000004	93518.51000000001	6287.560000000001	101.08

Figure 4: alt text

Komentarz:

- suma narastająca rośnie mniej więcej równomiernie - choć obserwujemy pewne odstające dni, w których przychód wynosi np. ~4 tysiące lub ~20 tysięcy, co generalny trend jest raczej stabilny (w większości dni wartości rosną o ~8-12 tysięcy). Co istotne, nawet po wystąpieniu przychodu dziennego w wysokości ~20k, wartości przychodu w kolejnych dniach wracają do normy.
- obswerwujemy kilka dni z wyraźną zmianą, przykładowo 15.01.2025 nastąpił wzrost dobowy o 168%, 19.03.2025 nastąpił wzrost o ~148%, a 05.06.2025 nastąpił spadek o ~65%.
- funckje okna były nam już znane wcześniej, ponieważ były one już przerabiane na poprzednich laboratoriach. Z tego względu nie sprawiły one większym problemów.

3. Segmentacja użytkowników - metoda RFM - 2 pkt

Dlaczego to robimy

RFM to prosta i skuteczna metoda segmentacji klientów stosowana w marketingu od dekad:

- Recency - jak dawno użytkownik ostatnio kupił?
- Frequency - ile razy kupił?
- Monetary - ile łącznie wydał?

Na tej podstawie dzielisz klientów na segmenty: najlepszych nagradzasz, uśpionych reaktywujesz, nowych zachęcasz do kolejnego zakupu.

Wybierz jedną bazę i wykonaj zadanie w niej

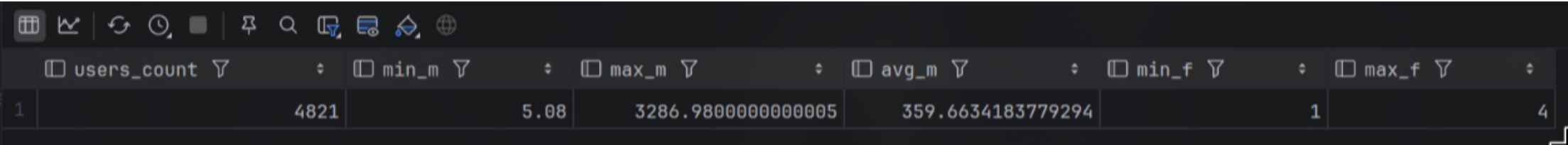
Zaznacz w sprawozdaniu, którą bazę wybrałeś.

Użyta baza: PostgreSQL

Krok 1 — oblicz R, F, M dla każdego użytkownika

```
-- Wersja PostgreSQL
-- Dla ClickHouse: zamień DATE_PART na dateDiff('day', ...)
-- patrz ściąga
WITH ref AS (
    SELECT MAX(event_time) AS ref_time
    FROM events
),
purchases AS (
    SELECT
        user_id,
        COUNT(*) AS frequency,
        SUM(price * quantity) AS monetary,
        MAX(event_time) AS last_purchase_time
    FROM events
    WHERE event_type = 'purchase'
    GROUP BY user_id
```

```
)
SELECT
  p.user_id,
  DATE_PART('day', ref.ref_time - p.last_purchase_time) AS recency,
  p.frequency,
  p.monetary
FROM purchases p
CROSS JOIN ref
ORDER BY monetary DESC;
```



	users_count	min_m	max_m	avg_m	min_f	max_f
1	4821	5.08	3286.9800000000005	359.6634183779294	1	4

Figure 5: alt text

Zanim przejdziesz dalej — sprawdź zakres wartości, żeby świadomie dobrać progi:

```
-- Poznaj rozkład danych przed doбором progów
WITH purchases AS (
  SELECT
    user_id,
    COUNT(*) AS frequency,
    SUM(price * quantity) AS monetary
  FROM events
  WHERE event_type = 'purchase'
  GROUP BY user_id
)
SELECT
  COUNT(*) AS users_count,
  MIN(monetary) AS min_m,
  MAX(monetary) AS max_m,
  AVG(monetary) AS avg_m,
  MIN(frequency) AS min_f,
  MAX(frequency) AS max_f
FROM purchases;
```

Output

Result 5

	user_id	recency	frequency	monetary
1	23810	80	2	3286.9800000000005
2	34269	27	2	2774.81
3	33378	59	2	2564.93
4	45959	61	2	2512.83
5	34262	146	1	2465.1499999999996
6	16444	41	1	2445.65
7	47090	117	1	2442.85
8	35823	161	1	2432.75
9	3940	14	1	2427.8
10	13037	88	1	2402.3500000000004
11	14089	56	1	2378.4
12	25552	2	1	2352.3500000000004
13	35735	142	1	2349.5
14	26366	103	1	2335.8
15	3042	60	1	2334.1
16	15277	21	1	2332.6499999999996
17	32253	69	3	
18	11051	176	1	

<<< < 1-500 of 501+ > >>>

Figure 6: alt text

Krok 2 — podział użytkowników na segmenty

Na podstawie wyników z kroku 1 zbuduj trzy segmenty według kolumny monetary. Progi dobierz samodzielnie na podstawie rozkładu danych z powyższego kroku.

```
-- Przykładowy fragment segmentacji
-- Dostosuj progi do swoich danych
CASE
  WHEN monetary >= <twój_próg_wysoki> THEN 'premium'
  WHEN monetary >= <twój_próg_średni> THEN 'standard'
  ELSE 'okazjonalny'
END AS segment
```

Dla każdego segmentu policz:

- liczbę użytkowników,
- łączny przychód segmentu,

- udział procentowy w całkowitym przychodzie wszystkich użytkowników.

W komentarzu napisz

- Jak dobrałeś progi i dlaczego - co konkretnie w danych na to wskazało?
- Czy mała grupa użytkowników generuje dużą część przychodu - jaki procent użytkowników i jaki procent przychodu?
- Czy widzisz coś zbliżonego do zasady Pareto?
- Co wyniki mówią o lojalności klientów tego sklepu?

Wyniki:

Segmentacja z procentowym udziałem w przychodzie:

```
WITH user_rfm AS (SELECT user_id,
                        COUNT(*)           AS frequency,
                        SUM(price * quantity) AS monetary,
                        MAX(event_time)      AS last_purchase_time
                    FROM events
                    WHERE event_type = 'purchase'
                    GROUP BY user_id),
segmentation AS (SELECT user_id,
                        monetary,
                        CASE
                            WHEN percent_rank() OVER (ORDER BY monetary) >= 0.90 THEN 'premium'
                            WHEN percent_rank() OVER (ORDER BY monetary) >= 0.50 THEN 'standard'
                            ELSE 'okazjonalny'
                        END AS segment
                    FROM user_rfm),
totals AS (SELECT SUM(monetary) AS total_revenue
            FROM user_rfm)
SELECT s.segment,
       COUNT(*)           AS users_count,
       ROUND(SUM(s.monetary)) AS segment_revenue,
       ROUND(SUM(s.monetary) * 100.0 / (SELECT total_revenue FROM totals)) AS revenue_share_pct
FROM segmentation s
GROUP BY s.segment
ORDER BY segment_revenue DESC;
```

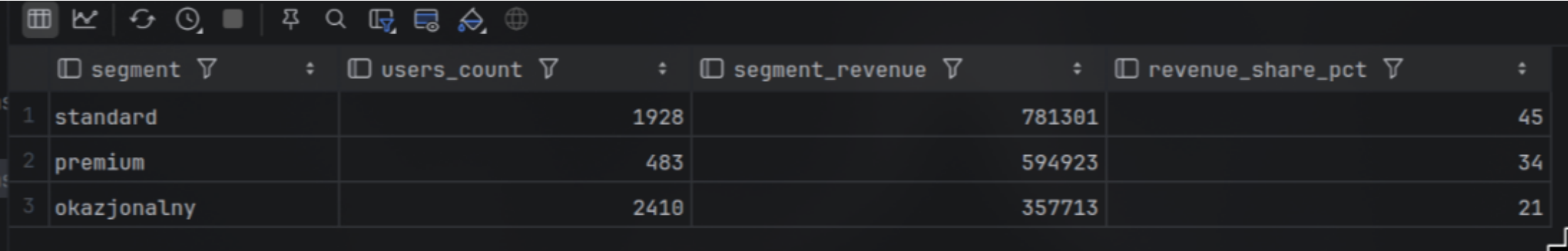


Figure 7: alt text

Komentarz:

- progi do segmentów zostały dobrane na podstawie percentyla - 90% dla premium, 50% dla standard. To pozwala na dynamiczny podział bez konieczności ręcznego ustalania progów kwotowych
- mała grupa użytkowników generuje dużą część przychodu - 10% użytkowników (segment premium) generuje 34% przychodu, podczas gdy 40% użytkowników (segment standard) generuje 45% przychodu, a pozostałe 50% użytkowników (segment okazjonalny) generuje 21% przychodu.
- nie ma ściśle zasady Pareto 80/20, jest to bardziej rozmyte, ale widać, że niewielka grupa użytkowników (premium) generuje znaczący procent przychodu.
- klienci sklepu raczej nie są lojalni - większość należy do segmentu okazjonalnego (50%), a tylko niewielka część to klienci premium. Sklep powinien skupić się na strategiach retencji i zwiększania wartości klientów standardowych, aby przesunąć ich do segmentu premium.

4. Benchmark i wnioski końcowe - 4 pkt

Dlaczego to robimy

Przez całe laboratorium pisałeś zapytania analityczne. Teraz zmierzysz, jak szybko obie bazy te zapytania wykonują - i odpowiesz na pytanie będące sednem kursu: **kiedy i dlaczego ClickHouse wygrywa z PostgreSQL?** To eksperyment: hipoteza, pomiar, wynik, interpretacja.

Zasady pomiaru

Dla każdego zapytania w każdej bazie:

- uruchom zapytanie 4 razy,

- **pierwszy wynik odrzuć** - jest rozgrzewkowy,
- zapisz 3 kolejne czasy,
- oblicz średnią.

☒ **ClickHouse cache - krytyczne:** przed każdą serią pomiarów uruchom SET use_query_cache = 0. Bez tego kolejne wykonania identycznego zapytania mogą zwrócić wynik z pamięci podręcznej zamiast policzyć go od nowa — czasy będą sztucznie niskie i nieporównywalne.

Trzy zapytania do zmierzenia

Użyj zapytań, które już napisałeś w zadaniach 1–3. Nie pisz nowych, benchmark ma sens tylko wtedy, gdy mierzysz coś, co już rozumiesz.

B1 - proste - proste zapytanie agregujące z jednym GROUP BY, np. przychód per kraj albo liczba zdarzeń per dzień – **może to być zadanie z wcześniejszych laboratoriów**

B2 - średnie - zapytanie z lejką konwersji z zadania 1

B3 - złożone - zapytanie z funkcją okna z zadania 2 lub segmentacja RFM z zadania 3.

Uwaga: jeśli zadania 2 lub 3 pisałeś tylko w ClickHouse, dostosuj zapytanie B3 do PostgreSQL przed pomiarem. Różnice składniowe znajdziesz w ściądze. To jest celowe zobaczysz, że logika jest taka sama, zmienia się tylko dialekt.

Zapytania:

```
-- Clickhouse

-- B1 - proste
set use_query_cache = 0;

select country, sum(price * quantity)
from events
group by country

-- B2 - srednie

set use_query_cache = 0;

with session_flags as (select session_id,
                             device,
                             countIf(event_type = 'view') > 0      as has_view,
                             countIf(event_type = 'add_to_cart') > 0 as has_cart,
                             countIf(event_type = 'purchase') > 0   as has_purchase
                             from events
                             group by device, session_id)

select device,
       countIf(has_view)                as view_sessions,
       countIf(has_cart)                as cart_sessions,
       countIf(has_purchase)            as purchase_sessions,
       countIf(has_cart) / nullIf(countIf(has_view), 0) as cart_per_view,
       countIf(has_purchase) / nullIf(countIf(has_cart), 0) as purchase_per_cart,
       countIf(has_purchase) / nullIf(countIf(has_view), 0) as purchase_per_view
from session_flags
group by device;

-- B3 - zlozone

set use_query_cache = 0;

with user_rfm as (select user_id,
                        count(*)      as frequency,
                        sum(price * quantity) as monetary,
                        max(event_time)  as last_purchase_time
                        from events
                        where event_type = 'purchase'
                        group by user_id),
segmentation as (select user_id,
                        monetary,
                        case
                            when percent_rank() over (order by monetary) >= 0.90 then 'premium'
                            when percent_rank() over (order by monetary) >= 0.50 then 'standard'
                            else 'okazjonalny'
                        end as segment
                        from user_rfm),
totals AS (select SUM(monetary) AS total_revenue
            from user_rfm)

select s.segment,
```

```
count(*) as users_count,
round(sum(s.monetary)) as segment_revenue,
round(SUM(s.monetary) * 100.0 / (select total_revenue from totals)) as revenue_share_pct
from segmentation s
group by s.segment
order by segment_revenue desc;
```

```
-- Postgres

-- B1 - proste
-- takie samo jak w clickhouse tylko bez "set use_query_cache = 0;"
```

```
-- B2 - srednie
with session_flags as (
  select
    session_id,
    device,
    (sum(case when event_type = 'view' then 1 else 0 end) > 0)::int AS has_view,
    (sum(case when event_type = 'add_to_cart' then 1 else 0 end) > 0)::int AS has_cart,
    (sum(case when event_type = 'purchase' then 1 else 0 end) > 0)::int AS has_purchase
  from events
  group by device, session_id
)
select
  device,
  sum(has_view) AS view_sessions,
  sum(has_cart) AS cart_sessions,
  sum(has_purchase) AS purchase_sessions,
  1.0 * sum(has_cart) / sum(has_view) AS cart_per_view,
  1.0 * sum(has_purchase) / sum(has_cart) AS purchase_per_cart,
  1.0 * sum(has_purchase) / sum(has_view) AS purchase_per_view
from session_flags
group by device;
```

```
-- B3 - zlozone
-- takie samo jak w clickhouse tylko bez "set use_query_cache = 0;"
```

Tabela wyników

Wypełnij poniższą tabelę. Czasy podaj w milisekundach.

Zapytanie	Charakt.	PG p.1	PG p.2	PG p.3	PG śr.	CH p.1	CH p.2	CH p.3	CH śr.
B1 — proste	group by + sum()	11ms	13ms	12ms	12ms	9ms	10ms	10ms	9.66ms
B2 — średnie	CTE + group by + wielokrotne zliczanie warunkowe	58ms	56ms	61ms	58.33ms	20ms	19ms	24ms	21ms
B3 — złożone	CTE + funkcje okna + group by	11ms	17ms	12ms	13.33ms	19ms	22ms	23ms	21.33ms

W kolumnie „Charakterystyka“ wpisz jednym zdaniem, co zapytanie robi: ile kolumn czyta czy używa GROUP BY, COUNT(DISTINCT), funkcji okna, CTE.

Analiza planu wykonania

Dla zapytania **B1** i **B3** uruchom:

```
-- PostgreSQL – wykonuje zapytanie i pokazuje realne czasy
EXPLAIN ANALYZE <zapytanie>;
```

```
-- ClickHouse – pokazuje plan bez wykonania
EXPLAIN <zapytanie>;
```

Nie jest wymagana pełna analiza techniczna. Napisz po **2–3 zdania** dla każdego z czterech planów (B1 w PG, B1 w CH, B3 w PG, B3 w CH):

- gdzie w planie widać agregację i sortowanie,
- czy plan B3 jest wyraźnie bardziej rozbudowany niż B1,
- czym plan ClickHouse różni się od planu PostgreSQL - na co zwróciłeś uwagę.

Rezultaty - plany wykonania

Postgres - plan wykonania B1

Operation	Params	Rows	Actual Rows	Total Cost	Actual Total Time	Startup Cost	Actual Startup Time	Raw Desc
⌵ ⇐ Select								Planning Time = 0....
⌵ Aggregate		10	10	3112.1	21.632	3112.0	21.63	Strategy = Hashe...
📊 Full Scan (Seq	table: events;	100000	100000	2112.0	7.956	0.0	0.046	Parent Relationshi...

Figure 8: alt text

- plan zapytania składa się z jednego pełnego skanu tabeli i kroku agregującego
- postgres zdecydował nie sortować danych na potrzeby grupowania.

Postgres - plan wykonania B3

Operation	Params	Rows	Actual Rows	Total Cost	Actual Total Time	Startup Cost	Actual Startup Time	Raw Desc
⌵ ⇐ Select								Planning Time = 0.1...
⌵ Sort		200	3	3193.82	16.469	3193.32	16.467	Parallel Aware = fal...
⌵ Aggregate		4839	4821	2486.29	12.377	2437.9	11.524	Strategy = Hashed;...
📊 Full Scan (Seq Scan)	table: events;	5060	5092	2362.0	8.512	0.0	0.023	Parent Relationship...
⌵ Aggregate		1	1	108.89	0.462	108.88	0.462	Strategy = Plain; Pa...
📊 Access (CTE Scan)	table: user_rfm;	4839	4821	96.78	0.215	0.0	0.0	Parent Relationship...
⌵ Aggregate		200	3	590.5	16.458	586.5	16.456	Strategy = Hashed;...
⌵ Transformation (Window		4839	4821	501.82	15.466	392.94	14.62	Parent Relationship...
⌵ Sort		4839	4821	405.04	14.315	392.94	14.139	Parent Relationship...
📊 Access (CTE S	table: user_rfm;	4839	4821	96.78	13.353	0.0	11.527	Parent Relationship...

Figure 9: alt text

- plan zapytania B3 jest zdecydowanie bardziej rozbudowany niż B1 i składa się z wielu etapów
- plan uwzględnia etap sortowania (konieczne przy wykorzystaniu funkcji okna) oraz wielokrotne skanowanie CTE
- pomimo zdecydowanie bardziej rozbudowanego zapytania koszt zapytania jest porównywalny z zapytaniem B1

Clickhouse - plan wykonania B1

Operation	Params	Rows	Total Cost	Startup Cost	Raw Desc
⌵ ⇐ Select					
⌵ Transformation (Expression)					Node Id = Expression_7; Descripti...
⌵ Unknown (Aggregating)					Node Id = Aggregating_3;
⌵ Transformation (Expression)					Node Id = Expression_6; Descripti...
Unknown (ReadFromMergeTree)					Node Id = ReadFromMergeTree_0...

📄 explain 🗑	
1	Expression ((Project names + Projection))
2	Aggregating
3	Expression ((Before GROUP BY + Change column names to column identifiers))
4	ReadFromMergeTree (ds_lab.events)

- bardzo prosty plan zapytania, opiera się głównie na odczycie kolumnowym poprzez operację ReadFromMergeTree, która pobiera z dysku wyłącznie niezbędne kolumny
- w przypadku tego zapytania Clickhouse również nie wykorzystuje sortowania, zamiast tego dane trafiają bezpośrednio do kroku Aggregating

Clickhouse - plan wykonania B3

Operation	Params	Rows	Total Cost	Startup Cost	Raw Desc
⌵ ⌵ Select					
⌵ Transformation (Expression)					Node Id = Expression_54; De...
⌵ ⌵ Unknown (Sorting)					Node Id = Sorting_42; Descri...
⌵ ⌵ Transformation (Expression)					Node Id = Expression_52; De...
⌵ ⌵ ⌵ Unknown (Aggregating)					Node Id = Aggregating_39;
⌵ ⌵ ⌵ Transformation (Expression)					Node Id = Expression_50; De...
⌵ ⌵ ⌵ ⌵ Unknown (Window)					Node Id = Window_34; Descri...
⌵ ⌵ ⌵ ⌵ Unknown (Sorting)					Node Id = Sorting_33; Descri...
⌵ ⌵ ⌵ ⌵ Transformation (Expression)					Node Id = Expression_47; Des...
⌵ ⌵ ⌵ ⌵ Unknown (Aggregating)					Node Id = Aggregating_28;
⌵ ⌵ ⌵ ⌵ Transformation (Expression)					Node Id = Expression_27; Des...
⌵ ⌵ ⌵ ⌵ Transformation (Expression)					Node Id = Expression_55; De...
⌵ ⌵ ⌵ ⌵ ⌵ Unknown (ReadFromMergeT					Node Id = ReadFromMergeTr...

	⌵ explain ⌵	
1	Expression ((Project names + (Before ORDER BY + Projection) [lifted up part]))	
2	Sorting (Sorting for ORDER BY)	
3	Expression ((Before ORDER BY + Projection))	
4	Aggregating	
5	Expression ((Before GROUP BY + (Change column names to column identifiers + (Project names + Projection))))	
6	Window (Window step for window 'ORDER BY __table2.monetary ASC RANGE BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING')	
7	Sorting (Sorting for window 'ORDER BY __table2.monetary ASC RANGE BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING')	
8	Expression ((Before WINDOW + (Change column names to column identifiers + (Project names + Projection))))	
9	Aggregating	
10	Expression (Before GROUP BY)	
11	Expression ((WHERE + Change column names to column identifiers))	
12	ReadFromMergeTree (ds_lab.events)	

- plan zapytania B3 jest zdecydowanie bardziej rozbudowany niż w przypadku zapytania B1 i składa się z wielu etapów
- Clickhouse traktuje całe zapytanie jako ciąg/pipeline wielu transformacji, tym samym nie tworząc tymczasowych struktur na dysku
- w planie zapytania widać wyraźny etap sortowania (Sorting), który jest konieczny ze względu na wykorzystanie funkcji okna

Komentarz:

Kluczowa różnica to sposób pobierania danych - w przypadku Postgresa skanowane są całe wiersze tabeli Seq Scan, podczas gdy w Clickhousie dane odczytywane są kolumnowo (ReadFromMergeTree, odczytywane są tylko potrzebne atrybuty). Inną istotną różnicą jest podejście do wieloetapowego przetwarzania - Postgres wyraźnie wyodrębnia pracę na strukturach tymczasowych (CTE Scan), podczas gdy Clickhouse buduje jeden długi, strumieniowy ciąg operacji w pamięci operacyjnej.

Refleksja końcowa – obowiązkowa

Na podstawie pomiarów i planów napisz 5–8 zdań odpowiadając na poniższe pytania. To jest najważniejszy element całego laboratorium.

Czy różnica czasu między bazami rosła wraz ze złożonością zapytania?

Różnica czasu między bazami niekoniecznie rosła wraz ze złożonością zapytania - w przypadku zapytania B2 Postgres okazał się ~3 razy wolniejszy, natomiast dla zapytania B3 zapytanie w Postgres było ~1.5 raza szybsze. Różnica ta wynika z charakteru poszczególnych zapytań - przykładowo, w przypadku zapytania B2 konieczne było wielokrotne zliczanie warunkowe, co w przypadku Postgresa oznaczało wielokrotne pełne skanowanie tabeli, podczas gdy Clickhouse mógł w wydajny sposób odczytać wyłącznie potrzebne kolumny.

Przy którym zapytaniu przewaga ClickHouse była największa i jak to tłumaczysz?

Przewaga ClickHouse była największa w przypadku zapytania B2, gdzie Postgres okazał się ~3 razy wolniejszy. Wynika to z faktu, że zapytanie to opiera się na agregacji wielu zdarzeń do poziomu sesji oraz wielokrotnego zliczania warunkowego, co idealnie wpasowuje się w silnik analityczny Clickhouse’a, który jest w stanie efektywnie sumować warunki logiczne z wybranych kolumn bez konieczności wczytywania pozostałych atrybutów.

Co plany wykonania mówią o tym, dlaczego ClickHouse jest szybszy dla tego rodzaju zapytań?

Plany wykonania pokazują 2 kluczowe różnice pomiędzy Clickhousem oraz Postgresem:

- Postgres musi zawsze skanować (i wczytywać) pełne wiersze (Seq Scan), podczas gdy ClickHouse ładuje z dysku wyłącznie niezbędne atrybuty (ReadFromMergeTree)
- Clickhouse przetwarza dane w jednym, ciągłym potoku operacji wykonywanych w pamięci, jednocześnie unikając kosztownego zapisywania wyników pośrednich (co ma miejsce w przypadku zapytań w Postgresie (CTE Scan))

Gdybyś był architektem systemu danych w firmie e-commerce obsługującej miliony zdarzeń dziennie - dla jakich zadań wybrałbyś ClickHouse, a dla jakich PostgreSQL? Uzasadnij konkretnie.

Prawdopodobnie wybrałbym PostgreSQL jako główną bazę transakcyjną do obsługi bieżących operacji dotyczących produktów i klientów, w której przechowywałbym wszystkie informacje o klientach, produktach i transakcjach/zamówieniach. Clickhouse’a użyłbym natomiast np. do przetrzymywania wszystkich danych odnośnie zdarzeń w aplikacji/systemie, dzięki czemu możliwe byłyby bardzo wydajne tworzenie raportów oraz analiza lejków konwersji (jednocześnie operacje te nie obciążałyby głównej bazy Postgres). W celu przeprowadzania dokładnych analiza dotyczących sprzedaży, rozważyłbym także cykliczne kopiowanie danych dotyczących zamówień (np. tych już zrealizowanych) z Postgresa do Clickhouse’a - dzięki temu mamy główne źródło prawdy w Postgresie, ale możemy w wydajny sposób tworzyć raporty na podstawie danych w Clickhouse.

Co jest oceniane

Element	Punkty
Zadanie 1 – lejek konwersji w ClickHouse + komentarz z wyjaśnieniem countIf	2
Zadanie 2 – funkcje okna: ranking i trend z LAG	2
Zadanie 3 – segmentacja RFM z własnym doborem progów	2
Zadanie 4 – pomiary benchmarkowe z wypełnioną tabelą	1
Zadanie 4 – analiza planów EXPLAIN	1
Zadanie 4 – refleksja końcowa	1
Jakość komentarzy i interpretacji we wszystkich zadaniach	1
Razem	10

Napisanie działającego kodu to warunek konieczny, nie wystarczający. Jeżeli uruchomiłeś zapytanie i dostałeś wynik, ale nie rozumiesz, co z niego wynika dla biznesu, oddałeś połowę zadania.